```
    "_id":[10,4,6,8]
    }
  }
}
```

Here, we use a `terms` query and set the array of document IDs in the `_id` field as our search criteria. Now, let's look at another term-level query type: the `exists` query.

9.5 The exists query

Sometimes, project documents have hundreds of fields. Fetching all the fields in a response is a waste of bandwidth, and knowing whether a field exists before attempting to fetch it is a good precheck. An `exists` query fetches documents for a given field if the field exists. For example, if we run the following query, we get a response containing the document because the document with the `title` field exists.

Listing 9.11 Running an `exists` query to check whether a field exists

```
GET movies/_search
{
  "query": {
    "exists": {
      "field": "title"
    }
  }
}
```

Defines the query type as exists

Provides the field that we want to check in the document

If the field doesn't exist, the query returns an empty `hits` array (`hits[]`). If you are curious, try this query with a nonexistent field like `title2`, and you'll see the empty array.

There's another subtle use case of an `exists` query: to retrieve all documents that *don't* have a particular field (a nonexistent field). For example, the next listing checks for all documents that aren't classified as confidential (assuming classified documents have a field called `confidential` set to `true`).

Listing 9.12 Finding non-confidential documents

```
PUT top_secret_files/_doc/1
{
  "code":"Flying Bird",
  "confidential":true
}

PUT top_secret_files/_doc/2
{
  "code":"Cold Rock"
}
GET top_secret_files/_search
{
  "query": {
    "bool": {
      "must_not": [{
```

Adds two documents, one with a confidential flag

Compound query that fetches documents without a confidential field